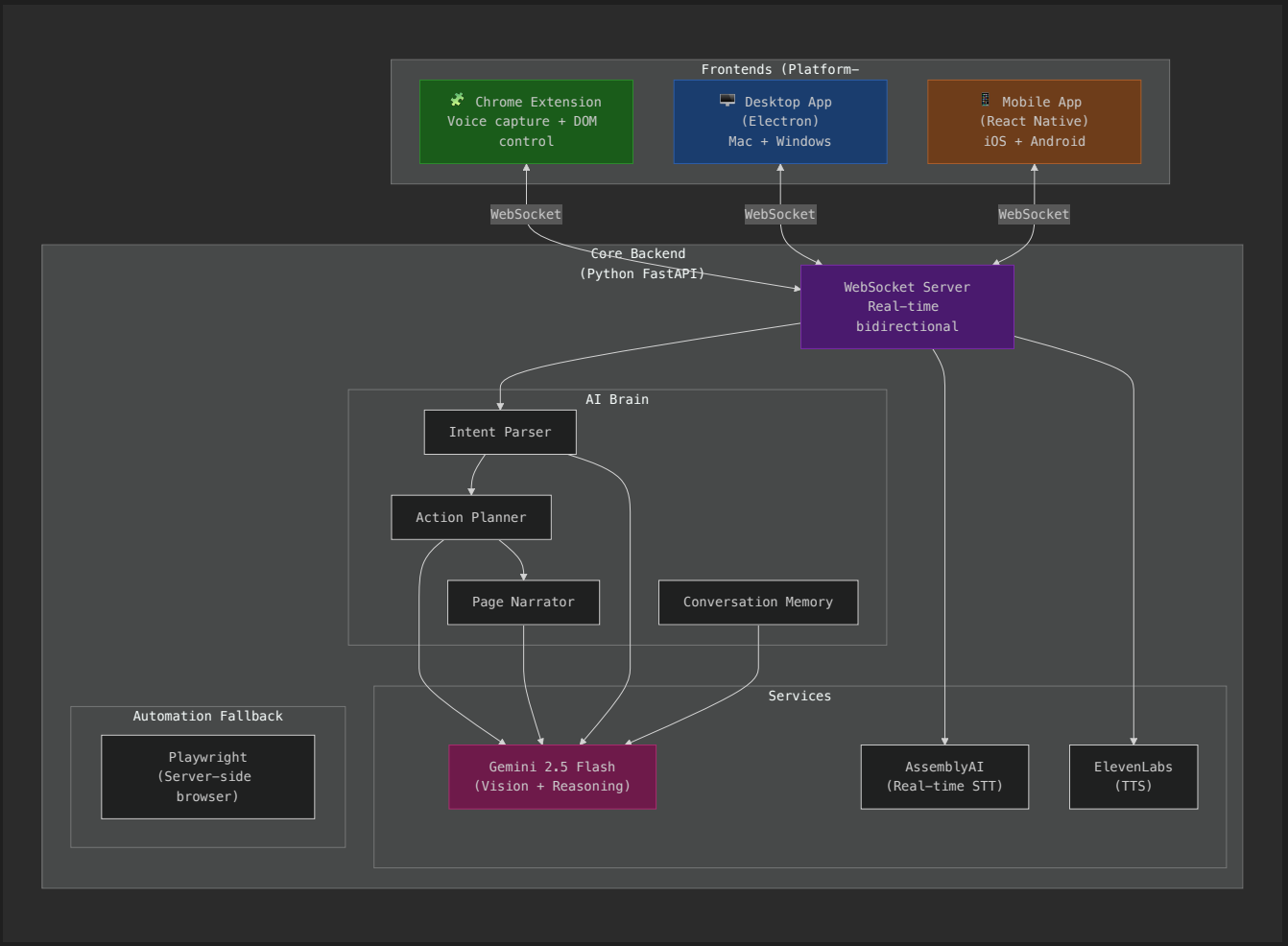
TernConnect — Multi-Platform AI Accessibility Agent

A voice-driven AI agent that gives blind users **full control** over web platforms — especially LMS systems — through natural speech. Deployable as a **Chrome extension**, **desktop app** (Mac + Windows), and **mobile app**.

Architecture Overview

The key insight: separate the **brain** (AI + automation) from the **interface** (voice I/O + platform). This lets one core power all platforms.



Why This Architecture?

Approach	Pros	Cons
Chrome Extension + Backend ✅	Controls user's real browser, uses their existing sessions/logins, fastest for web	Requires backend running
Playwright only	Works headlessly	Separate browser, can't reuse user's sessions
Desktop automation only	Controls any app	Fragile, coordinate-dependent, slow

Decision: Chrome extension is the **primary** interface. It directly manipulates the DOM of the page the user is on — no separate browser, no coordinate guessing, no login duplication.

User Review Required

> [!IMPORTANT]
> **Phase 1 scope**: We build the **Core Backend + Chrome Extension** first. Desktop and mobile come later as they wrap the same backend. Is this phasing acceptable?

> [!IMPORTANT]
> **Project location**: New project at `/Users/sunday/Documents/Project/TERNCONNECT/ternconnect/` with two sub-projects: `backend/` (Python) and `extension/` (TypeScript/JS). Confirm this path.

> [!WARNING]
> **Voice activation**: For the Chrome extension, the browser's Web Audio API handles mic capture. The extension will use a **keyboard shortcut** (configurable, default `Alt+T`) to start/stop listening. Always-listening requires the `"background"` permission and uses more STT credits. Which do you prefer?

Open Questions

> [!IMPORTANT]
> **Which LMS platforms are highest priority?** The AI-vision approach works on any LMS, but knowing your primary targets helps me write better prompts. Common ones:
>
> - Coursera, Udemy, edX, Khan Academy
> - Moodle, Canvas, Blackboard (institutional)
> - Custom platforms
> - All of the above equally

> [!IMPORTANT]
> **Quiz strategy**: Should the AI:
>
> - **Read-only** — read questions and options, user answers verbally, AI selects their answer
> - **Tutoring** — AI explains concepts and helps the user reason through answers
> - **Configurable** — user chooses mode per session

> [!IMPORTANT]
> **Backend deployment**: During development, the backend runs locally (`localhost:8000`). For production/mobile, it would need a cloud server. Should I design for:
>
> - **Local-only** (desktop + extension talk to localhost)
> - **Cloud-ready** (add auth, HTTPS, deployable to any VPS)
> - **Both** (local dev + cloud production config)

Proposed Changes

Project Structure

```
ternconnect/
├── backend/                          # Python FastAPI server (the brain)
│   ├── main.py                       # FastAPI app + WebSocket endpoints
│   ├── config.py                     # All configuration + env vars
│   ├── requirements.txt              # Python dependencies
│   ├── .env.example                  # API key template
│   │
│   ├── api/                          # WebSocket + REST endpoints
│   │   ├── __init__.py
│   │   ├── websocket_handler.py      # Real-time voice + action protocol
│   │   └── routes.py                 # REST endpoints (health, config)
│   │
│   ├── brain/                        # AI reasoning engine
│   │   ├── __init__.py
│   │   ├── gemini_client.py          # Gemini API (vision + text + streaming)
│   │   ├── intent_parser.py          # Speech → structured intent
│   │   ├── action_planner.py         # Intent + page state → DOM actions
│   │   ├── page_narrator.py          # Page state → spoken description
│   │   ├── conversation_memory.py    # Multi-turn context tracking
│   │   └── prompts.py                # All system prompts
│   │
│   ├── services/                     # External API integrations
│   │   ├── __init__.py
│   │   ├── assemblyai_stt.py         # Real-time speech-to-text
│   │   └── elevenlabs_tts.py         # Text-to-speech synthesis
│   │
│   └── models/                       # Shared data models
│       ├── __init__.py
│       ├── intents.py                # UserIntent, ActionPlan dataclasses
│       ├── actions.py                # BrowserAction, ActionResult
│       └── messages.py               # WebSocket message protocol
│
├── extension/                        # Chrome Extension (the hands)
│   ├── manifest.json                 # Chrome extension manifest v3
│   ├── package.json                  # Node dependencies (for build)
│   │
│   ├── background/                   # Service worker (always running)
│   │   └── service-worker.js         # WebSocket connection, audio routing
│   │
│   ├── content/                      # Content scripts (injected into pages)
│   │   ├── dom-controller.js         # Click, type, scroll, select on the page
│   │   ├── page-reader.js            # Extract accessibility tree + page state
│   │   └── content.css               # Minimal visual feedback styles
│   │
│   ├── popup/                        # Extension popup UI
│   │   ├── popup.html                # Settings + status panel
│   │   ├── popup.js                  # Popup logic
```

```
│   │   └── popup.css                    # Popup styles
│   │
│   ├── offscreen/                       # Offscreen document (audio processing)
│   │   ├── offscreen.html               # Required for audio APIs in MV3
│   │   └── offscreen.js                 # Mic capture + TTS playback
│   │
│   └── shared/                          # Shared utilities
│       ├── protocol.js                  # WebSocket message types (matches backend)
│       └── constants.js                 # Shared constants
│
└── README.md                            # Project documentation
```

Core Backend (Python FastAPI)

WebSocket Protocol

The backend and extension communicate via a **bidirectional WebSocket** with typed JSON messages:


```
Extension → Backend:
  { type: "audio_chunk", data: "<base64 PCM16>" }
  { type: "page_state", data: { url, title, accessibility_tree, screenshot_b64 } }
  { type: "action_result", data: { success, error, new_page_state } }

Backend → Extension:
  { type: "speak", data: { text, audio_b64 } }           # TTS audio to play
  { type: "execute_action", data: { action, selector, value } }  # DOM action
  { type: "request_page_state", data: {} }                # Ask for current page
  { type: "transcript_update", data: { text, is_final } } # Live STT feedback
  { type: "status", data: { state: "listening|thinking|acting|speaking" } }
```

[NEW] backend/main.py

FastAPI application entry point:

- Mounts WebSocket endpoint at `/ws`
- Health check at `/health`
- CORS middleware for extension origin
- Lifespan handler to initialize/cleanup AI clients

[NEW] backend/config.py

```python
# API Keys (from .env)
GEMINI_API_KEY: str
ASSEMBLYAI_API_KEY: str
ELEVENLABS_API_KEY: str
ELEVENLABS_VOICE_ID: str

# Models
GEMINI_MODEL = "gemini-2.5-flash"
ASSEMBLYAI_SPEECH_MODEL = "u3-rt-pro"
ELEVENLABS_MODEL = "eleven_flash_v2_5"

# Server
HOST = "127.0.0.1"
PORT = 8000
CORS_ORIGINS = ["chrome-extension://*"]

# AI behavior
CONVERSATION_HISTORY_MAX_TURNS = 20
NARRATION_VERBOSITY = "normal"  # brief | normal | detailed
ACTION_TIMEOUT_SECONDS = 10
MAX_RETRY_ATTEMPTS = 3
```

[NEW] backend/api/websocket_handler.py

The heart of the real-time communication. Manages a single user session:

```python
class SessionManager:
    """
    One session per connected client. Orchestrates the full loop:

    1. Receive audio chunks → stream to AssemblyAI
    2. When speech ends → parse intent via Gemini
    3. Request page state from extension
    4. Plan actions via Gemini (with vision)
    5. Send actions to extension for execution
    6. Narrate results via TTS → send audio back
    """

    async def handle_connection(self, websocket: WebSocket):
        await websocket.accept()
        self.speak("Tern Connect is ready. What would you like to do?")

        async for message in websocket.iter_json():
            match message["type"]:
                case "audio_chunk":
                    await self._handle_audio(message)
                case "page_state":
                    await self._handle_page_state(message)
                case "action_result":
                    await self._handle_action_result(message)
                case "listening_started":
                    await self._start_stt_session()
                case "listening_stopped":
                    await self._finalize_and_process()
```

[NEW] backend/brain/intent_parser.py

Classifies transcribed speech into structured intents. Uses Gemini with a specialized prompt.

```python
class IntentCategory(str, Enum):
    NAVIGATE = "navigate"       # Go somewhere: "open my courses", "go to linkedin"
    READ = "read"               # Read content: "read this page", "what does it say"
    INTERACT = "interact"       # Click/type: "click the login button", "type my email"
    COURSE = "course"           # Course-specific: "enroll", "start lesson 3", "next lesson"
    QUIZ = "quiz"               # Quiz-specific: "read question", "select answer B", "submit"
    SEARCH = "search"           # Find things: "search for python courses"
    AUTH = "auth"               # Login/signup: "log me in", "create an account"
    GENERAL = "general"         # Freeform: "what am I looking at?", "help"

@dataclass
class UserIntent:
    category: IntentCategory
    action: str                 # Specific action within category
    parameters: dict            # Extracted entities
    confidence: float           # 0-1 confidence score
    raw_transcript: str         # Original speech
```

Full intent mapping for LMS workflows:

User says	Category	Action	Parameters
"go to coursera"	navigate	open_url	{url: "coursera.org"}
"sign me up"	auth	signup	{}
"log in with my email"	auth	login	{method: "email"}
"search for machine learning courses"	search	search_courses	{query: "machine learning"}
"tell me about the first course"	read	describe_item	{position: 1}
"enroll in this course"	course	enroll	{}
"start the course"	course	start	{}
"next lesson"	course	next_lesson	{}
"read this page to me"	read	narrate_page	{}
"take the quiz"	quiz	start_quiz	{}
"read question 3"	quiz	read_question	{number: 3}
"select answer B"	quiz	select_answer	{answer: "B"}

"submit the quiz"	`quiz`	`submit`	`{}`
"what's my score?"	`quiz`	`read_results`	`{}`
"go back to the course"	`navigate`	`go_back`	`{}`

[NEW] backend/brain/action_planner.py

Takes a `UserIntent` + page state (accessibility tree + screenshot) → outputs precise DOM actions.

```python
@dataclass
class DOMAction:
    action: str             # "click" | "type" | "select" | "scroll" | "navigate" | "wait" | "read"
    selector: str | None    # CSS selector, ARIA label, or text content
    value: str | None       # Text to type, option to select, URL to navigate
    description: str        # Spoken to user: "clicking the enroll button"
    fallback_selector: str | None  # Backup selector if primary fails
```

The planner is **vision-powered**: it receives the page screenshot + accessibility tree and uses Gemini to determine the exact elements to interact with. This makes it **LMS-agnostic** — it works on Coursera, Udemy, Moodle, or any platform because it *sees* the page like a human would.

Key planning strategies:

- **Multi-step planning**: "Log me in" generates: click email field → type email → click password field → type password → click submit
- **Smart waiting**: After navigation, waits for page load before next action
- **Error recovery**: If a click fails, re-captures page state and tries an alternative selector
- **Confirmation before destructive actions**: "Are you sure you want to submit the quiz?"

[NEW] backend/brain/page_narrator.py

Converts raw page state into natural, spoken descriptions optimized for blind users.

Narration modes:

- **Overview**: "You're on Coursera's course catalog. There are 12 courses matching 'python'. The first is Python for Everybody by University of Michigan, rated 4.8 stars."
- **Focused**: "This is lesson 3: Variables and Data Types. The video is 12 minutes long. Below it is a transcript and a 'Next Lesson' button."
- **Quiz**: "Question 2 of 10. What is the output of print(2 ** 3)? A: 6. B: 8. C: 9. D: 23."
- **Result**: "You scored 8 out of 10 on the quiz. 80 percent. Questions 4 and 7 were incorrect."

Design for the ear, not the eye:

- Numbers spoken naturally ("four point eight stars" not "4.8 ★")
- Spatial cues ("at the top of the page", "the third item in the list")
- Progressive disclosure (overview first, details on request)

[NEW] backend/brain/conversation_memory.py

Maintains full conversation context across turns:

```python
class ConversationMemory:
    """
    Tracks:
    - Last N exchanges (user speech + AI response + page context)
    - Current page URL and state
    - Current workflow (e.g., "taking quiz on lesson 3")
    - User preferences learned during session
    - Credentials (in-memory only, never persisted)
    """
```

This enables natural follow-ups:

- User: "Search for python courses" → AI searches
- User: "Tell me about the third one" → AI knows context
- User: "Enroll in that one" → AI knows which course

[NEW] backend/brain/prompts.py

Centralized, carefully crafted system prompts. This is where the "intelligence" lives:

```python
SYSTEM_PROMPT_CORE = """
You are TernConnect, an AI accessibility assistant for blind users.
You help users navigate web platforms entirely through voice.

CRITICAL RULES:
- You ARE the user's eyes. Describe everything clearly and concisely.
- Speak naturally — your words are read aloud via TTS.
- Never say "I can see" — say "the page shows" or "there is"
- Use positional language: "first item", "at the top", "the button on the right"
- Be proactive: if you notice something useful, mention it
- Confirm before destructive actions (submit quiz, delete, purchase)
- If something goes wrong, explain clearly and suggest alternatives
"""

SYSTEM_PROMPT_ACTION_PLANNER = """
You are planning DOM actions for a browser automation system.
Given the user's intent and the current page state (accessibility tree + screenshot),
output a JSON array of actions to execute.

You MUST output valid CSS selectors or ARIA labels.
Prefer ARIA labels and roles over CSS classes (more stable across page updates).
Always include a human-readable description for each action.
"""

SYSTEM_PROMPT_LMS = """
Additional context: the user is navigating a Learning Management System.
Common patterns:
- Course catalogs have search bars, filters, and course cards
- Course pages have lessons/modules listed vertically
- Video lessons have play buttons, transcripts, and next/previous buttons
- Quizzes have numbered questions with radio buttons or checkboxes
- Progress is usually shown as a percentage or progress bar
"""
```

[NEW] backend/services/assemblyai_stt.py

Adapted from Clicky's `assemblyai_provider.py`:

- Same v3 WebSocket streaming protocol
- Audio chunks received via WebSocket from extension → forwarded to AssemblyAI
- Live transcript updates sent back to extension for feedback
- Final transcript triggers intent parsing

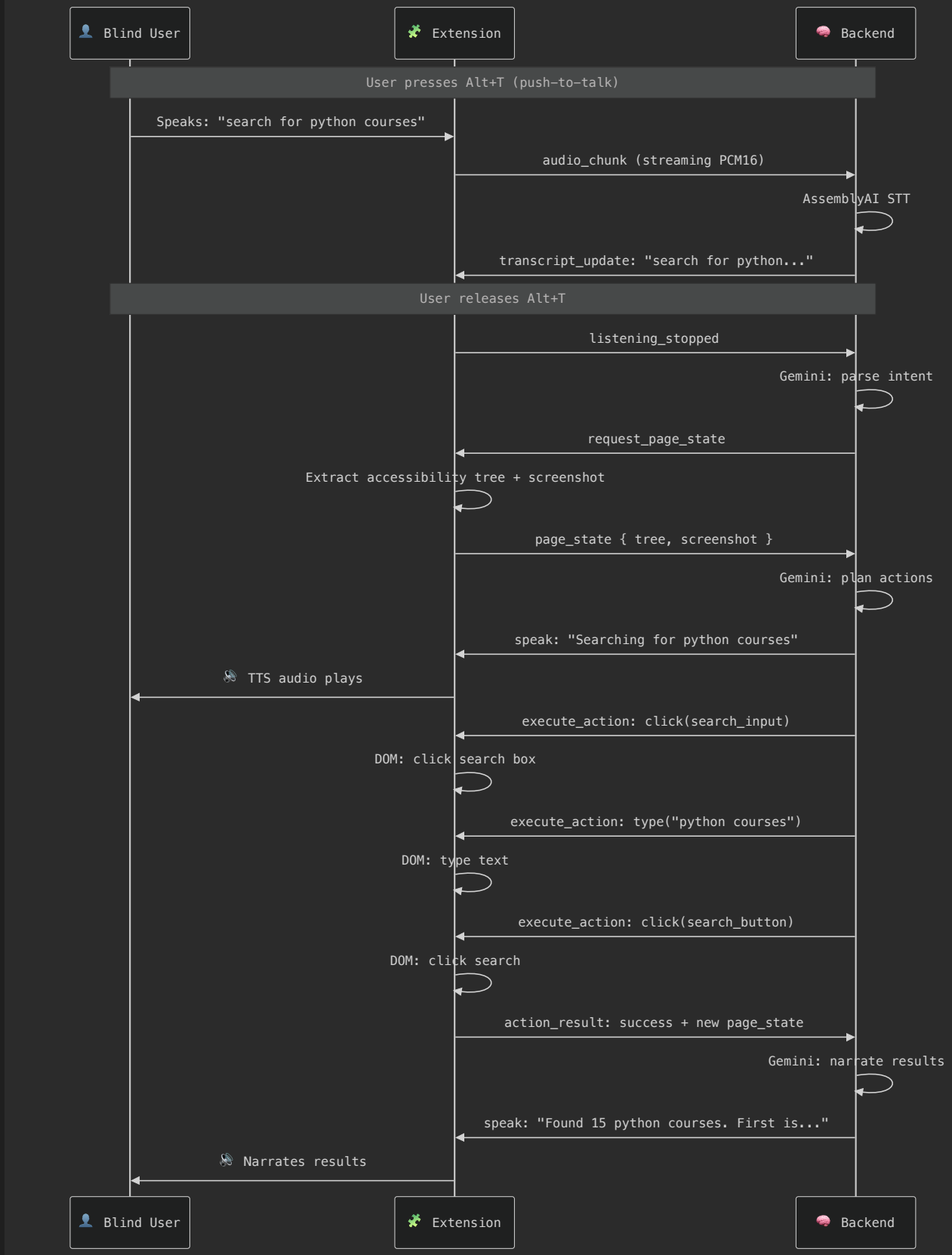
[NEW] backend/services/elevenlabs_tts.py

Adapted from Clicky's `elevenlabs_client.py`:

- Generates MP3 audio from text
- Returns base64-encoded audio to send via WebSocket
- Supports interruption (if user speaks while TTS is playing)

Chrome Extension

How It Works


Blind User
Extension
Backend
User presses Alt+T (push-to-talk)
Speaks: "search for python courses"
audio_chunk (streaming PCM16)
AssemblyAI STT
transcript_update: "search for python..."
User releases Alt+T
listening_stopped
Gemini: parse intent
request_page_state
Extract accessibility tree + screenshot
page_state { tree, screenshot }
Gemini: plan actions
speak: "Searching for python courses"
TTS audio plays
execute_action: click(search_input)
DOM: click search box
execute_action: type("python courses")
DOM: type text
execute_action: click(search_button)
DOM: click search
action_result: success + new page_state
Gemini: narrate results
speak: "Found 15 python courses. First is..."
Narrates results
Blind User
Extension
Backend

[NEW] extension/manifest.json

```json
{
  "manifest_version": 3,
  "name": "TernConnect",
  "version": "1.0.0",
  "description": "AI accessibility agent — navigate any website by voice",
  "permissions": [
    "activeTab",
    "scripting",
    "offscreen",
    "storage",
    "tabs"
  ],
  "host_permissions": ["<all_urls>"],
  "background": {
    "service_worker": "background/service-worker.js"
  },
  "content_scripts": [{
    "matches": ["<all_urls>"],
    "js": ["content/page-reader.js", "content/dom-controller.js"],
    "css": ["content/content.css"]
  }],
  "action": {
    "default_popup": "popup/popup.html",
    "default_icon": "icons/icon48.png"
  },
  "commands": {
    "toggle-listening": {
      "suggested_key": { "default": "Alt+T" },
      "description": "Start/stop voice input"
    }
  }
}
```

[NEW] extension/background/service-worker.js

The orchestrator inside the extension:

- Maintains WebSocket connection to backend (`ws://localhost:8000/ws`)
- Routes audio from offscreen document → backend
- Routes actions from backend → content scripts
- Handles keyboard shortcut registration
- Manages extension lifecycle

[NEW] extension/content/dom-controller.js

Injected into every web page. Executes DOM actions received from the backend:

```javascript
class DOMController {
  // Core actions
  click(selector)              // Find element + click it
  type(selector, text)         // Focus element + type text
  selectOption(selector, value) // Select dropdown option
  scroll(direction, amount)     // Scroll page or element
  navigate(url)                 // window.location navigation

  // Smart element finding (fallback chain)
  findElement(selector) {
    // 1. Try CSS selector
    // 2. Try ARIA label: [aria-label="..."]
    // 3. Try text content match
    // 4. Try role + name: [role="button"][name="..."]
    // 5. Try XPath as last resort
  }

  // Action feedback
  highlightElement(element)    // Brief visual flash (for sighted helpers)
  announceAction(description)  // SR-friendly announcement
}
```

[NEW] extension/content/page-reader.js

Extracts page state for the AI brain:

```javascript
class PageReader {
  // Full page analysis
  getPageState() {
    return {
      url: location.href,
      title: document.title,
      accessibility_tree: this.buildAccessibilityTree(),
      interactive_elements: this.getInteractiveElements(),
      headings: this.getHeadingStructure(),
      forms: this.getFormState(),
      text_content: this.getMainTextContent()
    }
  }

  // Accessibility tree builder
  buildAccessibilityTree(root = document.body) {
    // Walks the DOM and builds a simplified tree:
    // { role, name, value, children, selector, is_interactive }
    // Filters out decorative/hidden elements
    // Prioritizes ARIA roles and labels
  }

  // Screenshot via html2canvas or extension API
```

```
    async captureScreenshot() { ... }
  }
```

[NEW] extension/offscreen/offscreen.js

Chrome MV3 requires an offscreen document for audio APIs:

- Captures microphone audio via `navigator.mediaDevices.getUserMedia()`
- Encodes as PCM16 (16kHz mono) using AudioWorklet
- Streams chunks to service worker → backend
- Plays TTS audio received from backend
- Handles audio interruption (stop playback when user speaks)

[NEW] extension/popup/popup.html + popup.js + popup.css

Simple settings panel:

- Connection status indicator (connected/disconnected to backend)
- Backend URL configuration
- Voice shortcut display
- Narration verbosity toggle (brief/normal/detailed)
- "Start Backend" button (launches local Python server)

LMS Workflow: Complete Course Journey

Here's how a blind user takes a full course, from discovery to completion:

```
1. DISCOVERY
   User: "Find a good python course for beginners"
   → AI searches, reads results, recommends top-rated options

2. ENROLLMENT
   User: "Enroll in the first one"
   → AI clicks enroll, handles any signup/payment flow
   → "You're now enrolled in Python for Everybody"

3. STARTING
   User: "Start the course"
   → AI navigates to first lesson
   → "Lesson 1: Why We Program. This is a 15-minute video."

4. LEARNING
   User: "Play the video" / "Read the transcript"
   → AI controls video player or reads lesson text
   User: "Next lesson"
   → AI navigates forward, describes new content

5. QUIZZING
   User: "Take the quiz"
   → AI opens quiz, reads first question + all options
   User: "I think it's B"
   → AI selects B, confirms, moves to next question
   User: "Submit the quiz"
   → AI confirms, submits, reads the score
   → "You got 9 out of 10. Great job! Question 7 was incorrect."

6. PROGRESS
   User: "How far am I in the course?"
   → AI reads progress: "You've completed 4 of 12 modules. 33 percent."
   User: "Continue where I left off"
   → AI navigates to next unfinished lesson
```

Verification Plan

Automated Tests

```bash
# ---- Backend ----
cd /Users/sunday/Documents/Project/TERNCONNECT/ternconnect/backend
pip install -r requirements.txt
python -m pytest tests/ -v

# ---- Extension ----
cd /Users/sunday/Documents/Project/TERNCONNECT/ternconnect/extension
# Load as unpacked extension in Chrome for testing

# ---- Integration test ----
# 1. Start backend: python main.py
# 2. Load extension in Chrome
# 3. Navigate to a test LMS page
# 4. Press Alt+T and speak a command
```

Manual Verification

1. **Connection**: Extension popup shows "Connected" when backend is running
2. **Voice loop**: Press Alt+T → speak → hear TTS response
3. **Page reading**: Navigate to any page → "Read this page" → hear description
4. **LMS login**: Navigate to Coursera → "Log me in" → AI guides through login
5. **Course search**: "Search for python courses" → hear results narrated
6. **Quiz taking**: Open a quiz → AI reads questions → user answers → submit → hear score

Implementation Phases

Phase 1 — Core + Chrome Extension (BUILD NOW)

- ✅ FastAPI backend with WebSocket server
- ✅ Gemini-powered intent parsing + action planning + page narration
- ✅ AssemblyAI real-time STT integration
- ✅ ElevenLabs TTS integration
- ✅ Chrome extension: DOM controller + page reader + audio I/O
- ✅ Full LMS workflow: search → enroll → learn → quiz
- ✅ Conversation memory for natural follow-ups

Phase 2 — Polish + LinkedIn

- ⬜ LinkedIn handler (feed, posts, jobs, messaging)
- ⬜ Wake word detection ("Hey Tern")
- ⬜ Smarter error recovery and retry logic
- ⬜ User preference learning (remembers preferred LMS, verbosity)
- ⬜ Audio feedback sounds (beeps for state changes)

Phase 3 — Desktop App

- ⬜ Electron wrapper (bundles backend + extension-like UI)
- ⬜ System tray with status indicator
- ⬜ Auto-start backend on app launch
- ⬜ Mac + Windows installers

Phase 4 — Mobile App

- ⬜ React Native app with voice capture
- ⬜ Cloud-deployed backend for mobile connectivity
- ⬜ Adapted UI for mobile screen readers (VoiceOver/TalkBack)